UIPATH TRAINING

Complete Solutions for Scenarios 01–06

Activities • Variables • Arguments • Expressions • Expected Results • REFramework Customization

Important
All expressions below are written for a VB-based UiPath Windows project. Activity labels can vary slightly with installed package versions, but the workflow logic and values remain the same. Scenario 06 is written specifically as a queue-based REFramework Performer.

Document Map

Scenario	Solution Focus
01	Excel DataTable filter, count, total, average, write results
02	Excel-driven local web form automation with selectors and per-row status
03	PDF customer extraction + billing filters + averages + OMR calculations
04	Config-driven reusable workflow + timestamped output + email
05	Orchestrator Dispatcher + Performer + Queue + Asset + Switch + exceptions
06	Final Dispatcher + current REFramework Performer customization

REFramework verification note
The Scenario 06 guidance keeps the standard queue transaction lifecycle intact. The current UiPath documentation still describes REFramework as a State Machine template with prebuilt initialization, transaction retrieval, processing, and exception handling. The official UiPath/ReFrameWork repository was archived on 26 June 2026; its source confirms the standard Main.xaml state machine, queue-based GetTransactionData.xaml, root Process.xaml, and SetTransactionStatus.xaml. Newer packaged templates also include the consecutive-system-exception / job-fault controls introduced by UiPath. If your installed template contains additional framework fields, keep them unchanged unless explicitly listed below.

SCENARIO 01 — COMPLETE SOLUTION

Excel Data Processing Fundamentals

Blank Process / Sequence — do not use REFramework

A. Project Files

Item	Value
Input / Output workbook	S01_Excel_Transactions.xlsx
Source sheet	Transactions
Result sheet	Summary
Project type	Blank Process (VB / Windows)

B. Variables

Variable	Type	Default / Purpose
excelPath	String	Full path to S01_Excel_Transactions.xlsx
dtTransactions	System.Data.DataTable	All source records
dtFiltered	System.Data.DataTable	IT + Approved rows only
approvedITCount	Int32	Filtered record count
totalAmount	Double	Filtered Amount total
averageAmount	Double	Filtered Amount average

C. Main.xaml — Exact Activity Order

1. Assign — *Display Name: Set Excel Path*

Property / Field	Value
To	excelPath
Value	System.IO.Path.Combine(Environment.CurrentDirectory, "S01_Excel_Transactions.xlsx")

2. Log Message — *Display Name: Log Process Start*

Property / Field	Value
Level	Info
Message	"Scenario 01 started."

3. Use Excel File — *Display Name: Open Transaction Workbook*

Property / Field	Value
Excel file	excelPath
Save changes	True

3.1. Read Range — *Display Name: Read Transactions*

Property / Field	Value
Sheet	Transactions
Range	A3:G43
Has headers	True
Save to	dtTransactions

3.2. Filter Data Table — *Display Name: Filter IT Approved*

Property / Field	Value
Input DataTable	dtTransactions
Output DataTable	dtFiltered
Condition 1	Department = "IT"
Connector	AND
Condition 2	Status = "Approved"
Columns	Keep all columns

3.3. Assign — *Display Name: Count Approved IT Records*

Property / Field	Value
To	approvedITCount
Value	dtFiltered.Rows.Count

3.4. Assign — *Display Name: Calculate Total Amount*

Property / Field	Value
To	totalAmount
Value	If(dtFiltered.Rows.Count = 0, 0.0, dtFiltered.AsEnumerable().Sum(Function(r) Convert.ToDouble(r("Amount (OMR)"))))

3.5. Assign — *Display Name: Calculate Average Amount*

Property / Field	Value
To	averageAmount
Value	If(approvedITCount = 0, 0.0, totalAmount / approvedITCount)

3.6. Write Cell — *Display Name: Write Record Count*

Property / Field	Value
Value	approvedITCount
Destination	Summary!B4

3.7. Write Cell — *Display Name: Write Total Amount*

Property / Field	Value
Value	totalAmount
Destination	Summary!B5

3.8. Write Cell — *Display Name: Write Average Amount*

Property / Field	Value
Value	averageAmount
Destination	Summary!B6

3.9. Write DataTable to Excel — *Display Name: Write Filtered Rows*

Property / Field	Value
What to write	dtFiltered
Destination	Summary!A9
Exclude headers	True

The Summary sheet already contains headers in row 8, so exclude DataTable headers.

4. Log Message — *Display Name: Log Process End*

Property / Field	Value
Level	Info
Message	"Scenario 01 completed. Records=" + approvedITCount.ToString

D. Expected Answer

Output	Expected Value
Summary!B4	14
Summary!B5	3131.000
Summary!B6	223.643 (rounded to 3 decimals)

Filtered transactions
The correct filtered set contains 14 rows. The automation must calculate these values; do not hard-code them.

SCENARIO 02 — COMPLETE SOLUTION

Web / UI Customer Registration Automation

Excel + local HTML page + modern UI automation

A. Project Files

Item	Value
Excel	S02_Customer_Input.xlsx
Local web page	S02_Customer_Form.html
Sheet	Customers
Browser used in this solution	Microsoft Edge

B. Variables

Variable	Type	Purpose
excelPath	String	Excel file path
formPath	String	HTML file path
browserUrl	String	file:/// URL for local HTML
dtCustomers	DataTable	Customer rows
confirmationText	String	Text from #result
excelRowNumber	Int32	Physical Excel row to update

C. Main.xaml — Exact Activity Order

1. Assign — *Display Name: Set Excel Path*

Property / Field	Value
To	excelPath
Value	System.IO.Path.Combine(Environment.CurrentDirectory, "S02_Customer_Input.xlsx")

2. Assign — *Display Name: Set Form Path*

Property / Field	Value
To	formPath
Value	System.IO.Path.Combine(Environment.CurrentDirectory, "S02_Customer_Form.html")

3. Assign — *Display Name: Build Browser URL*

Property / Field	Value
To	browserUrl
Value	New System.Uri(formPath).AbsoluteUri

4. Use Excel File — *Display Name: Open Customer Input*

Property / Field	Value
Excel file	excelPath
Save changes	True

4.1. Read Range — *Display Name: Read Customer Data*

Property / Field	Value
Sheet	Customers
Range	A3:G18
Has headers	True
Save to	dtCustomers

4.2. Use Application/Browser — *Display Name: Open Training Customer Form*

Property / Field	Value
Browser	Microsoft Edge
URL	browserUrl
Open	Always / IfNotOpen
Close	At end of scope

4.2.1. For Each Row in Data Table — *Display Name: Process Customers*

Property / Field	Value
DataTable	dtCustomers
Current row	CurrentRow
Index	CurrentIndex

4.2.1.1. Assign — *Display Name: Calculate Excel Row*

Property / Field	Value
To	excelRowNumber
Value	CurrentIndex + 4

4.2.1.2. If — *Display Name: Check Process Flag*

Property / Field	Value
Condition	CurrentRow("Process").ToString.Trim.Equals("Yes", StringComparison.OrdinalIgnoreCase)

D. Inside IF — THEN

1. Try Catch — *Display Name: Process One Customer*

Property / Field	Value
Try	UI entry + submit + read result
Catch	System.Exception

1.1. Type Into — *Display Name: Enter Customer ID*

Property / Field	Value
Target	HTML input id = customerId
Text	CurrentRow("Customer ID").ToString
Empty field	True

1.2. Type Into — *Display Name: Enter Customer Name*

Property / Field	Value
Target	HTML input id = customerName
Text	CurrentRow("Customer Name").ToString
Empty field	True

1.3. Type Into — *Display Name: Enter Account Number*

Property / Field	Value
Target	HTML input id = accountNumber
Text	CurrentRow("Account Number").ToString
Empty field	True

1.4. Select Item — *Display Name: Select Department*

Property / Field	Value
Target	HTML select id = department
Item	CurrentRow("Department").ToString

1.5. Click — *Display Name: Submit Customer*

Property / Field	Value
Target	HTML button id = submitBtn

1.6. Check App State — *Display Name: Wait for Result*

Property / Field	Value
Target	HTML div id = result
Wait for	Element to appear
Timeout	5 seconds

1.7. Get Text — *Display Name: Read Confirmation*

Property / Field	Value
Target	HTML div id = result
Save to	confirmationText

1.8. If — *Display Name: Check Success*

Property / Field	Value
Condition	confirmationText.Trim.StartsWith("SUCCESS", StringComparison.OrdinalIgnoreCase)

1.8 THEN. Write Cell — *Display Name: Write Success*

Property / Field	Value
Value	"Success"
Destination	Customers!F + excelRowNumber.ToString

1.8 THEN. Write Cell — *Display Name: Write Confirmation*

Property / Field	Value
Value	confirmationText
Destination	Customers!G + excelRowNumber.ToString

1.8 ELSE. Write Cell — *Display Name: Write Failed*

Property / Field	Value
Value	"Failed"
Destination	Customers!F + excelRowNumber.ToString

1.8 ELSE. Write Cell — *Display Name: Write Error Confirmation*

Property / Field	Value
Value	confirmationText
Destination	Customers!G + excelRowNumber.ToString

1.9. Click — *Display Name: Clear Form*

Property / Field	Value
Target	HTML button id = clearBtn

E. Catch System.Exception

1. Write Cell — *Display Name: Write Failed*

Property / Field	Value
Value	"Failed"
Destination	Customers!F + excelRowNumber

2. Write Cell — *Display Name: Write Exception*

Property / Field	Value
Value	exception.Message
Destination	Customers!G + excelRowNumber

3. Log Message — *Display Name: Log Customer Error*

Property / Field	Value
Level	Error
Message	"Customer " + CurrentRow("Customer ID").ToString + ": " + exception.Message

F. Expected Answer

Check	Expected
Rows with Process = Yes	12
Rows with Process = No	C005, C011, C014 — Result/Confirmation remain blank
Processed rows	Result = Success
Confirmation format	SUCCESS | Customer ID | Customer Name | Account Number

SCENARIO 03 — COMPLETE SOLUTION

PDF + Excel Billing Processing

Modular process with invoked workflows

A. Required Project Structure

```
Main.xaml
Workflows\Extract_Customer.xaml
Workflows\Process_Billing.xaml
S03_Invoice_Source.pdf
S03_Billing_Processing.xlsx
```

B. Main.xaml Variables

Variable	Type
pdfPath	String
excelPath	String

1. Assign — *Display Name: Set PDF Path*

Property / Field	Value
To	pdfPath
Value	System.IO.Path.Combine(Environment.CurrentDirectory, "S03_Invoice_Source.pdf")

2. Assign — *Display Name: Set Excel Path*

Property / Field	Value
To	excelPath
Value	System.IO.Path.Combine(Environment.CurrentDirectory, "S03_Billing_Processing.xlsx")

3. Invoke Workflow File — *Display Name: Invoke Extract Customer*

Property / Field	Value
Workflow	Workflows\Extract_Customer.xaml
in_PdfPath	pdfPath
in_ExcelPath	excelPath

4. Invoke Workflow File — *Display Name: Invoke Process Billing*

Property / Field	Value
Workflow	Workflows\Process_Billing.xaml
in_ExcelPath	excelPath

C. Workflows\Extract_Customer.xaml

Argument	Direction	Type
in_PdfPath	In	String
in_ExcelPath	In	String

Variable	Type
pdfText	String
customerName	String

1. Read PDF Text — *Display Name: Read Invoice PDF*

Property / Field	Value
File name	in_PdfPath
Range	All
Output Text	pdfText

2. Assign — *Display Name: Extract Customer Name*

Property / Field	Value
To	customerName
Value	System.Text.RegularExpressions.Regex.Match(pdfText, "(?im)^\s*Customer\s*:\s*(.+?)\s*$").Groups(1).Value.Trim

3. If — *Display Name: Validate Customer*

Property / Field	Value
Condition	String.IsNullOrWhiteSpace(customerName)

THEN. Throw — *Display Name: Customer Missing*

Property / Field	Value
Exception	New BusinessRuleException("Customer Name was not found in the PDF.")

ELSE. Use Excel File — *Display Name: Open Billing Workbook*

Property / Field	Value
Excel file	in_ExcelPath

ELSE.1. Write Cell — *Display Name: Write Customer Name*

Property / Field	Value
Value	customerName
Destination	Invoice Register!E4

D. Workflows\Process_Billing.xaml

Argument	Direction	Type
in_ExcelPath	In	String

Variable	Type
dtBilling	DataTable
dtCurrentUSD	DataTable
dtISSR	DataTable
dtACQR	DataTable
dtBoth	DataTable
avgISSR	Double
avgACQR	Double
avgBoth	Double
bothHalf	Double

Variable	Type
finalISSR	Double
finalACQR	Double
fxRateObject	Object
fxRate	Double
issrOMR	Double
acqrOMR	Double
combinedUSD	Double
combinedOMR	Double

1. Use Excel File — *Display Name: Open Billing Workbook*

Property / Field	Value
Excel file	in_ExcelPath
Save changes	True

1.1. Read Range — *Display Name: Read Billing Data*

Property / Field	Value
Sheet	Billing Data
Range	A3:AD33
Has headers	True
Save to	dtBilling

1.2. Filter Data Table — *Display Name: Filter Current USD*

Property / Field	Value
Input	dtBilling
Output	dtCurrentUSD
Condition 1	Current or Previous = Current
AND	Billing Currency = USD

1.3. Filter Data Table — *Display Name: Filter ISSR*

Property / Field	Value
Input	dtCurrentUSD
Output	dtISSR
Condition	Type = ISSR

1.4. Filter Data Table — *Display Name: Filter ACQR*

Property / Field	Value
Input	dtCurrentUSD
Output	dtACQR
Condition	Type = ACQR

1.5. Filter Data Table — *Display Name: Filter Both*

Property / Field	Value
Input	dtCurrentUSD
Output	dtBoth
Condition	Type = Both

1.6. Assign — *Display Name: Calculate ISSR Average*

Property / Field	Value
To	avgISSR
Value	dtISSR.AsEnumerable().Average(Function(r)

Property / Field	Value
	Convert.ToDouble(r("Total")))

1.7. Assign — *Display Name: Calculate ACQR Average*

Property / Field	Value
To	avgACQR
Value	dtACQR.AsEnumerable().Average(Function(r) Convert.ToDouble(r("Total")))

1.8. Assign — *Display Name: Calculate Both Average*

Property / Field	Value
To	avgBoth
Value	dtBoth.AsEnumerable().Average(Function(r) Convert.ToDouble(r("Total")))

1.9. Assign — *Display Name: Divide Both By Two*

Property / Field	Value
To	bothHalf
Value	avgBoth / 2

1.10. Assign — *Display Name: Calculate Final ISSR*

Property / Field	Value
To	finalISSR
Value	avgISSR + bothHalf

1.11. Assign — *Display Name: Calculate Final ACQR*

Property / Field	Value
To	finalACQR
Value	avgACQR + bothHalf

1.12. Read Cell — *Display Name: Read FX Rate*

Property / Field	Value
Source	Invoice Register!I3
Output	fxRateObject

1.13. Assign — *Display Name: Convert FX Rate*

Property / Field	Value
To	fxRate
Value	Convert.ToDouble(fxRateObject)

1.14. Assign — *Display Name: Calculate ISSR OMR*

Property / Field	Value
To	issrOMR
Value	finalISSR * fxRate

1.15. Assign — *Display Name: Calculate ACQR OMR*

Property / Field	Value
To	acqrOMR
Value	finalACQR * fxRate

1.16. Assign — *Display Name: Calculate Combined USD*

Property / Field	Value
To	combinedUSD
Value	finalISSR + finalACQR

1.17. Assign — *Display Name: Calculate Combined OMR*

Property / Field	Value
To	combinedOMR
Value	issrOMR + acqrOMR

1.18. Write Cell — *Display Name: Write ISSR Average*

Property / Field	Value
Value	avgISSR
Destination	Invoice Register!I6

1.19. Write Cell — *Display Name: Write ACQR Average*

Property / Field	Value
Value	avgACQR
Destination	Invoice Register!I7

1.20. Write Cell — *Display Name: Write Both Average*

Property / Field	Value
Value	avgBoth
Destination	Invoice Register!I8

1.21. Write Cell — *Display Name: Write ISSR Both Half*

Property / Field	Value
Value	bothHalf
Destination	Invoice Register!J6

1.22. Write Cell — *Display Name: Write ACQR Both Half*

Property / Field	Value
Value	bothHalf
Destination	Invoice Register!J7

1.23. Write Cell — *Display Name: Write Final ISSR*

Property / Field	Value
Value	finalISSR
Destination	Invoice Register!K6

1.24. Write Cell — *Display Name: Write Final ACQR*

Property / Field	Value
Value	finalACQR
Destination	Invoice Register!K7

1.25. Write Cell — *Display Name: Write ISSR OMR*

Property / Field	Value
Value	issrOMR
Destination	Invoice Register!L6

1.26. Write Cell — *Display Name: Write ACQR OMR*

Property / Field	Value
Value	acqrOMR
Destination	Invoice Register!L7

1.27. Write Cell — *Display Name: Write Combined USD*

Property / Field	Value
Value	combinedUSD
Destination	Invoice Register!K9

1.28. Write Cell — *Display Name: Write Combined OMR*

Property / Field	Value
Value	combinedOMR
Destination	Invoice Register!L9

E. Expected Answer

Result	Expected
Customer Name	Al Noor Trading LLC
ISSR Average	1750.000
ACQR Average	1950.000
Both Average	1300.000
Both / 2	650.000
ISSR Final USD	2400.000
ACQR Final USD	2600.000
ISSR OMR	924.000
ACQR OMR	1001.000
Combined USD	5000.000
Combined OMR	1925.000

SCENARIO 04 — COMPLETE SOLUTION

Config-Driven Report, File Management and Email

Reusable workflows + clean template + timestamped output

Email assumption
This solution assumes Microsoft Outlook Desktop is already configured on the training machine. Use Desktop Outlook App + Send Email is therefore used. If your environment uses another approved mail connector, replace only the email workflow; the rest of the solution is unchanged.

A. Project Structure

```
Main.xaml
Config.xlsx
Data\Input\S04_Invoice_Source.pdf
Data\Input\S04_Billing_Data.xlsx
Data\Output\
Templates\S04_Report_Template.xlsx
Workflows\Init_Config.xaml
Workflows\Create_Output_File.xaml
Workflows\Process_Data.xaml
Workflows\Send_Email.xaml
```

B. Main.xaml Variables

Variable	Type
Config	Dictionary(Of String,String)
outputExcelPath	String

C. Main.xaml — Order

1. Invoke Workflow File — *Display Name: Initialization*

Property / Field	Value
Workflow	Workflows\Init_Config.xaml
out_Config	Config

2. Invoke Workflow File — *Display Name: Create Output File*

Property / Field	Value
Workflow	Workflows\Create_Output_File.xaml
in_Config	Config
out_OutputExcelPath	outputExcelPath

3. Invoke Workflow File — *Display Name: Process Data*

Property / Field	Value
Workflow	Workflows\Process_Data.xaml
in_Config	Config
in_OutputExcelPath	outputExcelPath

4. Invoke Workflow File — *Display Name: Send Email*

Property / Field	Value
Workflow	Workflows\Send_Email.xaml
in_Config	Config
in_OutputExcelPath	outputExcelPath

5. Log Message — *Display Name: Process Completed*

Property / Field	Value
Level	Info
Message	"Scenario 04 completed. Output: " + outputExcelPath

D. Workflows\Init_Config.xaml

Argument	Direction	Type
out_Config	Out	Dictionary(Of String,String)

Variable	Type
configPath	String
dtConfig	DataTable

1. Assign — *Display Name: Set Config Path*

Property / Field	Value
To	configPath
Value	System.IO.Path.Combine(Environment.CurrentDirectory, "Config.xlsx")

2. Use Excel File — *Display Name: Open Config*

Property / Field	Value
Excel file	configPath

2.1. Read Range — *Display Name: Read Settings*

Property / Field	Value
Sheet	Settings
Range	A:B
Has headers	True
Save to	dtConfig

3. Assign — *Display Name: Create Config Dictionary*

Property / Field	Value
To	out_Config
Value	dtConfig.AsEnumerable().Where(Function(r) Not String.IsNullOrWhiteSpace(r("Name").ToString)).ToDictionary(Function(r) r("Name").ToString.Trim, Function(r) r("Value").ToString.Trim)

E. Workflows\Create_Output_File.xaml

Argument	Direction	Type
in_Config	In	Dictionary(Of String,String)

Argument	Direction	Type
out_OutputExcelPath	Out	String

Variable	Type
templatePath	String
outputRoot	String
runFolder	String
outputFileName	String

1. Assign — *Display Name: Set Template Path*

Property / Field	Value
To	templatePath
Value	Path.Combine(Environment.CurrentDirectory, in_Config("TemplateFolder"), in_Config("TemplateFileName"))

2. Assign — *Display Name: Set Output Root*

Property / Field	Value
To	outputRoot
Value	Path.Combine(Environment.CurrentDirectory, in_Config("OutputRootFolder"))

3. Assign — *Display Name: Build Run Folder*

Property / Field	Value
To	runFolder
Value	Path.Combine(outputRoot, Now.ToString("yyyy"), Now.ToString("MM"), Now.ToString("dd"), "Run_" + Now.ToString("HHmmss"))

4. Create Folder — *Display Name: Create Run Folder*

Property / Field	Value
Folder	runFolder

5. Assign — *Display Name: Create Output File Name*

Property / Field	Value
To	outputFileName
Value	in_Config("OutputFilePrefix") + "_" + Now.ToString("yyyyMMdd_HHmmss") + ".xlsx"

6. Assign — *Display Name: Set Output Excel Path*

Property / Field	Value
To	out_OutputExcelPath
Value	Path.Combine(runFolder, outputFileName)

7. Copy File — *Display Name: Copy Clean Template*

Property / Field	Value
From	templatePath
To	out_OutputExcelPath
Overwrite	False

F. Workflows\Process_Data.xaml

Argument	Direction	Type
in_Config	In	Dictionary(Of String,String)
in_OutputExcelPath	In	String

Variable	Type
pdfPath	String
billingPath	String
pdfText	String
customerName	String
dtBilling	DataTable
dtCurrentUSD	DataTable
dtISSR	DataTable
dtACQR	DataTable
dtBoth	DataTable
avgISSR	Double
avgACQR	Double
avgBoth	Double
bothHalf	Double
finalISSR	Double
finalACQR	Double
fxRateObj	Object
fxRate	Double
issrOMR	Double
acqrOMR	Double
combinedUSD	Double
combinedOMR	Double

1. Assign — *Display Name: Set PDF Path*

Property / Field	Value
To	pdfPath
Value	Path.Combine(Environment.CurrentDirectory, in_Config("InputFolder"), in_Config("PDFFileName"))

2. Assign — *Display Name: Set Billing Path*

Property / Field	Value
To	billingPath
Value	Path.Combine(Environment.CurrentDirectory, in_Config("InputFolder"), in_Config("BillingFileName"))

3. Read PDF Text — *Display Name: Read Invoice PDF*

Property / Field	Value
File name	pdfPath
Output	pdfText

4. Assign — *Display Name: Extract Customer*

Property / Field	Value
To	customerName
Value	Regex.Match(pdfText, "(?im)^\s*Customer\s*:\s*(.+?)\s*$").Groups(1).Value.Trim

5. If — *Display Name: Validate Customer*

Property / Field	Value
Condition	String.IsNullOrWhiteSpace(customerName)

THEN. Throw — *Display Name: Missing Customer*

Property / Field	Value
Exception	New BusinessRuleException("Customer Name was not found in the PDF.")

6. Use Excel File — *Display Name: Read Billing Source*

Property / Field	Value
Excel file	billingPath

6.1. Read Range — *Display Name: Read Billing Data*

Property / Field	Value
Sheet	in_Config("BillingSheet")
Range	A3:F33
Has headers	True
Save to	dtBilling

7. Filter Data Table — *Display Name: Current USD*

Property / Field	Value
Input	dtBilling
Output	dtCurrentUSD
Conditions	Current or Previous = in_Config("FilterStatus") AND Billing Currency = in_Config("FilterCurrency")

8. Filter Data Table — *Display Name: ISSR*

Property / Field	Value
Input	dtCurrentUSD
Output	dtISSR
Condition	Type = in_Config("TypeISSR")

9. Filter Data Table — *Display Name: ACQR*

Property / Field	Value
Input	dtCurrentUSD
Output	dtACQR
Condition	Type = in_Config("TypeACQR")

10. Filter Data Table — *Display Name: Both*

Property / Field	Value
Input	dtCurrentUSD
Output	dtBoth
Condition	Type = in_Config("TypeBoth")

11. Assign — *Display Name: Average ISSR*

Property / Field	Value
To	avgISSR
Value	dtISSR.AsEnumerable().Average(Function(r) Convert.ToDouble(r("Total")))

12. Assign — *Display Name: Average ACQR*

Property / Field	Value
To	avgACQR
Value	dtACQR.AsEnumerable().Average(Function(r) Convert.ToDouble(r("Total")))

13. Assign — *Display Name: Average Both*

Property / Field	Value
To	avgBoth
Value	dtBoth.AsEnumerable().Average(Function(r) Convert.ToDouble(r("Total")))

14. Assign — *Display Name: Both / 2*

Property / Field	Value
To	bothHalf
Value	avgBoth / 2

15. Assign — *Display Name: Final ISSR USD*

Property / Field	Value
To	finalISSR
Value	avgISSR + bothHalf

16. Assign — *Display Name: Final ACQR USD*

Property / Field	Value
To	finalACQR
Value	avgACQR + bothHalf

17. Use Excel File — *Display Name: Open Output Workbook*

Property / Field	Value
Excel file	in_OutputExcelPath
Save changes	True

17.1. Read Cell — *Display Name: Read FX Rate*

Property / Field	Value
Source	Invoice Register!B9
Output	fxRateObj

17.2. Assign — *Display Name: Convert FX*

Property / Field	Value
To	fxRate
Value	Convert.ToDouble(fxRateObj)

17.3. Assign — *Display Name: ISSR OMR*

Property / Field	Value
To	issrOMR
Value	finalISSR * fxRate

17.4. Assign — *Display Name: ACQR OMR*

Property / Field	Value
To	acqrOMR
Value	finalACQR * fxRate

17.5. Assign — *Display Name: Combined USD*

Property / Field	Value
To	combinedUSD
Value	finalISSR + finalACQR

17.6. Assign — *Display Name: Combined OMR*

Property / Field	Value
To	combinedOMR
Value	issrOMR + acqrOMR

17.7. Write Cell — *Display Name: Write Customer Name*

Property / Field	Value
Value	customerName
Destination	Invoice Register!B4

17.8. Write Cell — *Display Name: Write Run Date/Time*

Property / Field	Value
Value	Now.ToString("dd/MM/yyyy HH:mm:ss")
Destination	Invoice Register!B5

17.9. Write Cell — *Display Name: Write Source File*

Property / Field	Value
Value	Path.GetFileName(billingPath)
Destination	Invoice Register!B6

17.10. Write Cell — *Display Name: Write ISSR Average*

Property / Field	Value
Value	avgISSR
Destination	Invoice Register!E4

17.11. Write Cell — *Display Name: Write ACQR Average*

Property / Field	Value
Value	avgACQR
Destination	Invoice Register!F4

17.12. Write Cell — *Display Name: Write Both Average*

Property / Field	Value
Value	avgBoth
Destination	Invoice Register!G4

17.13. Write Cell — *Display Name: Write ISSR Both Half*

Property / Field	Value
Value	bothHalf
Destination	Invoice Register!E5

17.14. Write Cell — *Display Name: Write ACQR Both Half*

Property / Field	Value
Value	bothHalf
Destination	Invoice Register!F5

17.15. Write Cell — *Display Name: Write ISSR Final USD*

Property / Field	Value
Value	finalISSR
Destination	Invoice Register!E6

17.16. Write Cell — *Display Name: Write ACQR Final USD*

Property / Field	Value
Value	finalACQR
Destination	Invoice Register!F6

17.17. Write Cell — *Display Name: Write Combined USD*

Property / Field	Value
Value	combinedUSD
Destination	Invoice Register!H6

17.18. Write Cell — *Display Name: Write ISSR OMR*

Property / Field	Value
Value	issrOMR
Destination	Invoice Register!E7

17.19. Write Cell — *Display Name: Write ACQR OMR*

Property / Field	Value
Value	acqrOMR
Destination	Invoice Register!F7

17.20. Write Cell — *Display Name: Write Combined OMR*

Property / Field	Value
Value	combinedOMR
Destination	Invoice Register!H7

17.21. Write Cell — *Display Name: Write Processing Status*

Property / Field	Value
Value	"Completed"
Destination	Invoice Register!B10

G. Workflows\Send_Email.xaml

Argument	Direction	Type
in_Config	In	Dictionary(Of String,String)
in_OutputExcelPath	In	String

1. Use Desktop Outlook App — *Display Name: Use Outlook*

Property / Field	Value
Account	Default Outlook profile

1.1. Send Email — *Display Name: Send Generated Report*

Property / Field	Value
To	in_Config("EmailTo")
Subject	in_Config("EmailSubject")
Body	"Please find the generated UiPath training report attached."
Attachments	New String() {in_OutputExcelPath}

H. Expected Answer

Result	Expected
Customer Name	Muscat Professional Services LLC
ISSR Average	1589.000
ACQR Average	1487.400
Both Average	1806.714
Both / 2	903.357
ISSR Final USD	2492.357
ACQR Final USD	2390.757
ISSR OMR	959.558
ACQR OMR	920.442
Combined USD	4883.114
Combined OMR	1879.999

SCENARIO 05 — COMPLETE SOLUTION

Orchestrator Queue — Dispatcher and Performer

Two separate projects; manual queue lifecycle; Switch required

A. Orchestrator Setup

Object	Value
Queue	TrainingInvoiceQueue
Text/Number Asset	TrainingFXRate = 0.385
Input file	Data\S05_Queue_Input.xlsx
Output template	Data\S05_Result_Template.xlsx

B. Project A — S05_Dispatcher

Variable	Type
Config	Dictionary(Of String,String)
dtConfig	DataTable
dtInput	DataTable
configPath	String
inputPath	String
queueCount	Int32

1. Assign — *Display Name: Set Config Path*

Property / Field	Value
To	configPath
Value	Path.Combine(Environment.CurrentDirectory, "Config.xlsx")

2. Use Excel File — *Display Name: Read Config*

Property / Field	Value
Excel file	configPath

2.1. Read Range — *Display Name: Read Settings*

Property / Field	Value
Sheet	Settings
Range	A:B
Has headers	True
Save to	dtConfig

3. Assign — *Display Name: Create Config Dictionary*

Property / Field	Value
To	Config
Value	dtConfig.AsEnumerable().Where(Function(r) Not String.IsNullOrWhiteSpace(r("Name").ToString)).ToDictionary(Function(r) r("Name").ToString.Trim, Function(r) r("Value").ToString.Trim)

4. Assign — *Display Name: Set Input Path*

Property / Field	Value
To	inputPath
Value	Path.Combine(Environment.CurrentDirectory, Config("InputFile"))

5. Use Excel File — *Display Name: Read Queue Input*

Property / Field	Value
Excel file	inputPath

5.1. Read Range — *Display Name: Read Transactions*

Property / Field	Value
Sheet	Config("InputSheet")
Range	A3:H38
Has headers	True
Save to	dtInput

6. Assign — *Display Name: Reset Queue Count*

Property / Field	Value
To	queueCount
Value	0

7. For Each Row in Data Table — *Display Name: Add Queue Items*

Property / Field	Value
DataTable	dtInput
Current row	CurrentRow

7.1. Add Queue Item — *Display Name: Add Training Transaction*

Property / Field	Value
Queue Name	Config("QueueName")
Reference	CurrentRow("Transaction ID").ToString
Priority	Normal
ItemInformation	New Dictionary(Of String,Object) From {{"TransactionID", CurrentRow("Transaction ID").ToString}, {"InvoiceID", CurrentRow("Invoice ID").ToString}, {"CustomerName", CurrentRow("Customer Name").ToString}, {"Currency", CurrentRow("Currency").ToString}, {"Amount", Convert.ToDouble(CurrentRow("Amount"))}, {"Type", CurrentRow("Type").ToString}, {"CurrentOrPrevious", CurrentRow("Current or Previous").ToString}, {"PriorityText", CurrentRow("Priority").ToString}}

7.2. Assign — *Display Name: Increment Count*

Property / Field	Value
To	queueCount
Value	queueCount + 1

8. Log Message — *Display Name: Dispatcher Complete*

Property / Field	Value
Level	Info
Message	"Queue items added: " + queueCount.ToString

C. Project B — S05_Performer

Variable	Type
Config	Dictionary(Of String,String)
transactionItem	UiPath.Core.QueueItem
continueProcessing	Boolean
fxAssetObject	Object
fxRate	Double
resultPath	String
resultRow	Int32
transactionID	String
invoiceID	String
customerName	String
currency	String
typeValue	String
currentOrPrevious	String
amount	Double
amountOMR	Double
processingStatus	String

1. Read Config — *Display Name: Use same Config loading logic as Dispatcher*

Property / Field	Value
Output	Config

2. Get Asset — *Display Name: Read Training FX Rate*

Property / Field	Value
Asset Name	Config("FXAssetName")
Output	fxAssetObject

3. Assign — *Display Name: Convert FX Rate*

Property / Field	Value
To	fxRate
Value	Convert.ToDouble(fxAssetObject)

4. Assign — *Display Name: Set Result Path*

Property / Field	Value
To	resultPath
Value	Path.Combine(Environment.CurrentDirectory, Config("ResultFile"))

5. If — *Display Name: Delete Old Result If Exists*

Property / Field	Value
Condition	File.Exists(resultPath)

THEN. Delete File — *Display Name: Delete Previous Result*

Property / Field	Value
Path	resultPath

6. Copy File — *Display Name: Create Result Workbook*

Property / Field	Value
From	Path.Combine(Environment.CurrentDirectory, Config("ResultTemplate"))
To	resultPath
Overwrite	False

7. Assign — *Display Name: Set First Result Row*

Property / Field	Value
To	resultRow
Value	4

8. Assign — *Display Name: Start Loop*

Property / Field	Value
To	continueProcessing
Value	True

9. While — *Display Name: Process Queue*

Property / Field	Value
Condition	continueProcessing

9.1. Get Transaction Item — *Display Name: Get Next Queue Item*

Property / Field	Value
Queue Name	Config("QueueName")
Transaction Item	transactionItem

9.2. If — *Display Name: No More Items*

Property / Field	Value
Condition	transactionItem Is Nothing

THEN. Assign — *Display Name: Stop Loop*

Property / Field	Value
To	continueProcessing
Value	False

D. Performer — ELSE: Process Current Queue Item

1. Try Catch — *Display Name: Process Transaction*

Property / Field	Value
Try	Extract + validate + Switch + write + success status
Catch 1	BusinessRuleException
Catch 2	System.Exception

1.1. Assign — *Display Name: Extract transactionID*

Property / Field	Value
To	transactionID
Value	transactionItem.SpecificContent("TransactionID").ToString

1.2. Assign — *Display Name: Extract invoiceID*

Property / Field	Value
To	invoiceID
Value	transactionItem.SpecificContent("InvoiceID").ToString

1.3. Assign — *Display Name: Extract customerName*

Property / Field	Value
To	customerName
Value	transactionItem.SpecificContent("CustomerName").ToString

1.4. Assign — *Display Name: Extract currency*

Property / Field	Value
To	currency
Value	transactionItem.SpecificContent("Currency").ToString.Trim.ToUpper

1.5. Assign — *Display Name: Extract amount*

Property / Field	Value
To	amount
Value	Convert.ToDouble(transactionItem.SpecificContent("Amount"))

1.6. Assign — *Display Name: Extract typeValue*

Property / Field	Value
To	typeValue
Value	transactionItem.SpecificContent("Type").ToString

1.7. Assign — *Display Name: Extract currentOrPrevious*

Property / Field	Value
To	currentOrPrevious
Value	transactionItem.SpecificContent("CurrentOrPrevious").ToString

1.8. If — *Display Name: Validate Current*

Property / Field	Value
Condition	Not currentOrPrevious.Equals("Current", StringComparison.OrdinalIgnoreCase)

THEN. Throw — *Display Name: Previous Transaction*

Property / Field	Value
Exception	New BusinessRuleException("Previous transaction is not eligible for processing.")

1.9. If — *Display Name: Validate Amount*

Property / Field	Value
Condition	amount <= 0

THEN. Throw — *Display Name: Invalid Amount*

Property / Field	Value
Exception	New BusinessRuleException("Amount must be greater than 0.")

1.10. Switch<String> — *Display Name: Convert Currency*

Property / Field	Value
Expression	currency
TypeArgument	String

Case "USD". Assign — *Display Name: Convert USD*

Property / Field	Value
To	amountOMR
Value	amount * fxRate

Case "OMR". Assign — *Display Name: Keep OMR*

Property / Field	Value
To	amountOMR
Value	amount

Default. Throw — *Display Name: Unsupported Currency*

Property / Field	Value
Exception	New BusinessRuleException("Unsupported currency: " + currency)

1.11. Assign — *Display Name: Set Success Status*

Property / Field	Value
To	processingStatus
Value	"Success"

1.12. Use Excel File — *Display Name: Write Successful Result*

Property / Field	Value
Excel file	resultPath

. Write Cell — *Display Name: Write A*

Property / Field	Value
Value	transactionID
Destination	Results!A + resultRow.ToString

. Write Cell — *Display Name: Write B*

Property / Field	Value
Value	invoiceID
Destination	Results!B + resultRow.ToString

. Write Cell — *Display Name: Write C*

Property / Field	Value
Value	customerName
Destination	Results!C + resultRow.ToString

. Write Cell — *Display Name: Write D*

Property / Field	Value
Value	amount
Destination	Results!D + resultRow.ToString

. Write Cell — *Display Name: Write E*

Property / Field	Value
Value	currency
Destination	Results!E + resultRow.ToString

. Write Cell — *Display Name: Write F*

Property / Field	Value
Value	amountOMR
Destination	Results!F + resultRow.ToString

. Write Cell — *Display Name: Write G*

Property / Field	Value
Value	processingStatus
Destination	Results!G + resultRow.ToString

. Write Cell — *Display Name: Write H*

Property / Field	Value
Value	Now.ToString("dd/MM/yyyy HH:mm:ss")
Destination	Results!H + resultRow.ToString

1.13. Set Transaction Status — *Display Name: Mark Successful*

Property / Field	Value
Transaction Item	transactionItem
Status	Successful

1.14. Assign — *Display Name: Next Result Row*

Property / Field	Value
To	resultRow
Value	resultRow + 1

E. Catch BusinessRuleException

1. Assign — *Display Name: Set BRE Status*

Property / Field	Value
To	processingStatus
Value	"Business Exception: " + businessException.Message

2. Write Result Row — *Display Name: Write failure row*

Property / Field	Value
Amount OMR	Leave blank / 0 as agreed
Status	processingStatus
Row	resultRow

3. Set Transaction Status — *Display Name: Mark Business Failed*

Property / Field	Value
Transaction Item	transactionItem
Status	Failed
ErrorType	Business
Reason	businessException.Message

4. Assign — *Display Name: Next Result Row*

Property / Field	Value
To	resultRow
Value	resultRow + 1

F. Catch System.Exception

1. Assign — *Display Name: Set System Status*

Property / Field	Value
To	processingStatus
Value	"System Exception: " + exception.Message

2. Write Result Row — *Display Name: Write system failure row*

Property / Field	Value
Status	processingStatus
Row	resultRow

3. Set Transaction Status — *Display Name: Mark Application Failed*

Property / Field	Value
Transaction Item	transactionItem
Status	Failed
ErrorType	Application
Reason	exception.Message

4. Assign — *Display Name: Next Result Row*

Property / Field	Value
To	resultRow
Value	resultRow + 1

G. Expected Queue Outcome

Metric	Expected
Input queue items	35
Successful	28
Business Exceptions	7

Transaction	Expected Business Exception
TX-0006	Previous transaction
TX-0009	Amount must be greater than 0
TX-0012	Previous transaction
TX-0018	Previous transaction
TX-0022	Amount must be greater than 0
TX-0024	Previous transaction
TX-0030	Previous transaction (also contains EUR, but Current/Previous validation occurs first)

Switch requirement
The Switch<String> on Currency is mandatory in this solution: USD → multiply by FX asset, OMR → keep amount, Default → BusinessRuleException.

SCENARIO 06 — COMPLETE SOLUTION

Final REFramework Invoice Automation

Dispatcher + queue-based REFramework Performer — current-template customization

Do not rebuild REFramework
Keep the framework state machine, its transitions, SetTransactionStatus logic, retry handling, screenshot handling, and standard exception catches. Add only the business-specific logic described below. Do NOT add a separate Set Transaction Status activity inside Process.xaml: the framework already updates Queue Items as Successful, Business Exception, or System/Application Exception after Process.xaml.

A. Orchestrator Setup

Object	Value / Setting
Queue	FinalInvoiceQueue
Queue Auto Retry	1 retry for training
Asset	FinalTrainingFXRate = 0.385
Performer	Use REFramework project template
Browser	Microsoft Edge

B. S06_Dispatcher — Exact Solution

Variable	Type
Config	Dictionary(Of String,String)
dtConfig	DataTable
dtTransactions	DataTable
inputPath	String
addedCount	Int32

1. Read Config.xlsx — *Display Name: Load Settings into Config*

Property / Field	Value
Path	Path.Combine(Environment.CurrentDirectory, "Config.xlsx")

2. Assign — *Display Name: Set Input Path*

Property / Field	Value
To	inputPath
Value	Path.Combine(Environment.CurrentDirectory, Config("InputTransactions"))

3. Use Excel File — *Display Name: Read Final Transactions*

Property / Field	Value
Excel file	inputPath

3.1. Read Range — *Display Name: Read Transactions*

Property / Field	Value
Sheet	Transactions
Range	A3:H15

Property / Field	Value
Has headers	True
Save to	dtTransactions

4. For Each Row in Data Table — *Display Name: Add Final Queue Items*

Property / Field	Value
DataTable	dtTransactions

4.1. Add Queue Item — *Display Name: Add Final Transaction*

Property / Field	Value
Queue Name	Config("QueueName") or literal FinalInvoiceQueue if Dispatcher Config uses that key
Reference	CurrentRow("Transaction ID").ToString
ItemInformation	New Dictionary(Of String,Object) From {{"TransactionID", CurrentRow("Transaction ID").ToString}, {"InvoiceID", CurrentRow("Invoice ID").ToString}, {"CustomerID", CurrentRow("Customer ID").ToString}, {"PDFFileName", CurrentRow("PDF File Name").ToString}, {"Currency", CurrentRow("Currency").ToString}, {"Amount", Convert.ToDouble(CurrentRow("Amount"))}, {"Type", CurrentRow("Type").ToString}, {"CurrentOrPrevious", CurrentRow("Current or Previous").ToString}}

C. REFramework — What Exists Already and What to Change

File / Area	Action for Scenario 06
Main.xaml State Machine	KEEP. Do not rebuild states or transitions.
Data\Config.xlsx	EDIT / ADD process settings, queue name, constants, and FX Asset mapping.
Framework\InitAllSettings.xaml	KEEP. No custom business logic.
Framework\InitAllApplications.xaml	EDIT: create output copy, load Customer Master, prepare portal URL, open browser.
Framework\KillAllProcesses.xaml	EDIT only for the dedicated training robot: kill msedge if required.
Framework\GetTransactionData.xaml	KEEP Get Transaction Item; EDIT only TransactionID/Field assignments for logging.
Process.xaml	EDIT: all per-transaction business logic goes here via reusable invoked workflows.
Framework\SetTransactionStatus.xaml	KEEP. No manual status logic inside Process.xaml.
Framework\TakeScreenshot.xaml	KEEP.
Framework\CloseAllApplications.xaml	EDIT: close Edge gracefully.
End Process state in Main.xaml	ADD one guarded Invoke Finalize_Run.xaml before/around final cleanup; keep existing framework fault logic.
Tests folder	KEEP; optional to add tests after process works.

D. Data\Config.xlsx — Exact Changes

Settings sheet

Name	Value	Purpose
OrchestratorQueueName	FinalInvoiceQueue	Default Get Transaction Item queue
logF_BusinessProcessName	S06_Final_Invoice_REFramework	Business process log field
CustomerMaster	Data\Input\S06_Customer_Master.xlsx	Customer master path
PdfFolder	Data\Input\PDF	Invoice PDF folder
PortalFile	S06_Invoice_Verification_Portal.html	Local verification portal
TemplatePath	Templates\S06_Final_Report_Template.xlsx	Clean output template
OutputRootFolder	Data\Output	Run output root
OutputFilePrefix	Final_Invoice_Report	Generated output prefix

Name	Value	Purpose
EmailTo	training.recipient@company.com	Training recipient
EmailSubject	UiPath REFramework Final Project Result	Final email subject
MaxRetryNumber	0	Use Orchestrator Queue retry instead of framework robot retry

Constants sheet

Name	Value	Purpose
MaxConsecutiveSystemExceptions	3	Stop after 3 consecutive system exceptions; use the row already present in current generated template if available
ShouldMarkJobAsFaulted	TRUE	Mark initialization failure / max consecutive system failure as Faulted when supported by the current template

Assets sheet

Name	Asset	Orchestrator Asset Folder	Description
FXRate	FinalTrainingFXRate		USD to OMR training rate

Config behavior
After InitAllSettings.xaml, the non-credential asset is available as Config("FXRate"). Use Convert.ToDouble(in_Config("FXRate")) in Process.xaml. Do not hard-code 0.385 in the Performer.

E. Main.xaml — Variables

Variable	Action
TransactionItem (QueueItem)	KEEP type as QueueItem
SystemError (Exception)	KEEP
BusinessRuleException	KEEP
TransactionNumber (Int32)	KEEP
Config (Dictionary<String,Object>)	KEEP
RetryNumber (Int32)	KEEP
TransactionField1 / TransactionField2 / TransactionID	KEEP and populate in GetTransactionData.xaml
TransactionData	KEEP
ConsecutiveSystemExceptions (if present)	KEEP; do not remove

No new Main variable required
Use runtime keys inside the Config dictionary for OutputExcelPath, CustomerMasterDT, and PortalUrl. This avoids adding new arguments to every framework state.

F. Framework\InitAllSettings.xaml

Change required: NONE
Do not rewrite this workflow. The framework already loads Settings / Constants and Orchestrator assets into Config. If an asset listed in Config is missing, current REFramework behavior is designed to fail initialization rather than continue with an empty value.

G. Framework\InitAllApplications.xaml — Add These Activities

Argument	Direction	Type
in_Config	In	Dictionary(Of String,Object) — already exists

Variable	Type
outputRoot	String
runFolder	String
outputFileName	String
outputExcelPath	String
customerMasterPath	String
dtCustomerMaster	DataTable
portalPath	String
portalUrl	String

1. Assign — *Display Name: Set Output Root*

Property / Field	Value
To	outputRoot
Value	Path.Combine(Environment.CurrentDirectory, in_Config("OutputRootFolder").ToString)

2. Assign — *Display Name: Build Run Folder*

Property / Field	Value
To	runFolder
Value	Path.Combine(outputRoot, Now.ToString("yyyy"), Now.ToString("MM"), Now.ToString("dd"), "Run_" + Now.ToString("HHmmss"))

3. Create Folder — *Display Name: Create Run Folder*

Property / Field	Value
Folder	runFolder

4. Assign — *Display Name: Build Output File Name*

Property / Field	Value
To	outputFileName
Value	in_Config("OutputFilePrefix").ToString + "_" + Now.ToString("yyyyMMdd_HHmmss") + ".xlsx"

5. Assign — *Display Name: Build Output Excel Path*

Property / Field	Value
To	outputExcelPath
Value	Path.Combine(runFolder, outputFileName)

6. Copy File — *Display Name: Copy Clean Report Template*

Property / Field	Value
From	Path.Combine(Environment.CurrentDirectory, in_Config("TemplatePath").ToString)
To	outputExcelPath
Overwrite	False

7. Assign — *Display Name: Set Customer Master Path*

Property / Field	Value
To	customerMasterPath
Value	Path.Combine(Environment.CurrentDirectory, in_Config("CustomerMaster").ToString)

8. Use Excel File — *Display Name: Read Customer Master*

Property / Field	Value
Excel file	customerMasterPath

8.1. Read Range — *Display Name: Load Customer Master*

Property / Field	Value
Sheet	Customer Master
Range	A3:C15
Has headers	True
Save to	dtCustomerMaster

9. Assign — *Display Name: Store Output Path in Config*

Property / Field	Value
To	in_Config("OutputExcelPath")
Value	outputExcelPath

10. Assign — *Display Name: Store Customer Master in Config*

Property / Field	Value
To	in_Config("CustomerMasterDT")
Value	dtCustomerMaster

11. Assign — *Display Name: Set Portal Path*

Property / Field	Value
To	portalPath
Value	Path.Combine(Environment.CurrentDirectory, in_Config("PortalFile").ToString)

12. Assign — *Display Name: Build Portal URL*

Property / Field	Value
To	portalUrl
Value	New Uri(portalPath).AbsoluteUri

13. Assign — *Display Name: Store Portal URL*

Property / Field	Value
To	in_Config("PortalUrl")
Value	portalUrl

14. Use Application/Browser — *Display Name: Open Verification Portal*

Property / Field	Value
Browser	Microsoft Edge
URL	portalUrl

Property / Field	Value
Open	Always / IfNotOpen
Close	Never — framework closes in CloseAllApplications.xaml

H. Framework\KillAllProcesses.xaml

Training VM only
Add Kill Process with ProcessName = "msedge" only if this is a dedicated training/robot machine. Do not kill a user's browser on a shared desktop. If your environment does not permit process killing, leave this workflow empty and rely on CloseAllApplications.xaml.

I. Framework\CloseAllApplications.xaml

1. Use Application/Browser — *Display Name: Attach to Verification Portal*

Property / Field	Value
Browser	Microsoft Edge
URL / target	in_Config("PortalUrl").ToString
Open	Never / Attach

1.1. Close Window / Close Tab — *Display Name: Close Training Portal*

Property / Field	Value
Target	Current Edge window / tab

J. Framework\GetTransactionData.xaml — Keep Queue Activity; Change Logging Fields

Keep this
Keep the existing Get Transaction Item activity with Queue Name = in_Config("OrchestratorQueueName").ToString and output = out_TransactionItem.

1. Assign — *Display Name: Transaction ID*

Property / Field	Value
To	out_TransactionID
Value	out_TransactionItem.Reference

2. Assign — *Display Name: Transaction Field 1*

Property / Field	Value
To	out_TransactionField1
Value	out_TransactionItem.SpecificContent("InvoiceID").ToString

3. Assign — *Display Name: Transaction Field 2*

Property / Field	Value
To	out_TransactionField2
Value	out_TransactionItem.SpecificContent("CustomerID").ToString

Do not change
Do not change out_TransactionItem type. Do not manually increment TransactionNumber. Do not add Set Transaction Status here.

K. Create Reusable Workflows for Process.xaml

1) Workflows\Read_Invoice_PDF.xaml

Argument	Direction	Type
in_PdfPath	In	String
out_PdfCustomerID	Out	String
out_PdfCustomerName	Out	String

Variable	Type
pdfText	String

1. Read PDF Text — *Display Name: Read Transaction PDF*

Property / Field	Value
File name	in_PdfPath
Output	pdfText

2. Assign — *Display Name: Extract PDF Customer ID*

Property / Field	Value
To	out_PdfCustomerID
Value	Regex.Match(pdfText, "(?im)^\s*Customer ID\s*:\s*(.+?)\s*$").Groups(1).Value.Trim

3. Assign — *Display Name: Extract PDF Customer Name*

Property / Field	Value
To	out_PdfCustomerName
Value	Regex.Match(pdfText, "(?im)^\s*Customer\s*:\s*(.+?)\s*$").Groups(1).Value.Trim

2) Workflows\Validate_Customer.xaml

Argument	Direction	Type
in_CustomerMasterDT	In	DataTable
in_QueueCustomerID	In	String
in_PdfCustomerID	In	String
in_PdfCustomerName	In	String
out_MasterCustomerName	Out	String
out_IsValid	Out	Boolean
out_ErrorMessage	Out	String

Variable	Type
masterRow	DataRow

1. Assign — *Display Name: Find Customer Master Row*

Property / Field	Value
To	masterRow

Property / Field	Value
Value	in_CustomerMasterDT.AsEnumerable().FirstOrDefault(Function(r) r("Customer ID").ToString.Trim.Equals(in_QueueCustomerID.Trim, StringComparison.OrdinalIgnoreCase))

2. If — *Display Name: Customer Not in Master*

Property / Field	Value
Condition	masterRow Is Nothing

THEN. Assign — *Display Name: Invalid*

Property / Field	Value
To	out_IsValid
Value	False

THEN. Assign — *Display Name: Error Message*

Property / Field	Value
To	out_ErrorMessage
Value	"Customer ID not found in Customer Master: " + in_QueueCustomerID

ELSE. Assign — *Display Name: Master Customer Name*

Property / Field	Value
To	out_MasterCustomerName
Value	masterRow("Customer Name").ToString.Trim

ELSE. If — *Display Name: PDF Customer ID Mismatch*

Property / Field	Value
Condition	Not in_PdfCustomerID.Trim.Equals(in_QueueCustomerID.Trim, StringComparison.OrdinalIgnoreCase)

THEN. Assign — *Display Name: Invalid ID*

Property / Field	Value
To	out_IsValid
Value	False

THEN. Assign — *Display Name: ID Error*

Property / Field	Value
To	out_ErrorMessage
Value	"PDF Customer ID does not match Queue Customer ID."

ELSE. If — *Display Name: PDF Customer Name Mismatch*

Property / Field	Value
Condition	Not in_PdfCustomerName.Trim.Equals(out_MasterCustomerName.Trim, StringComparison.OrdinalIgnoreCase)

THEN. Assign — *Display Name: Invalid Name*

Property / Field	Value
To	out_IsValid
Value	False

THEN. Assign — *Display Name: Name Error*

Property / Field	Value
To	out_ErrorMessage
Value	"PDF Customer Name does not match Customer Master."

FINAL ELSE. Assign — *Display Name: Valid Customer*

Property / Field	Value
To	out_IsValid
Value	True

FINAL ELSE. Assign — *Display Name: Clear Error*

Property / Field	Value
To	out_ErrorMessage
Value	String.Empty

3) Workflows\Verify_Invoice_Portal.xaml

Argument	Direction	Type
in_PortalUrl	In	String
in_InvoiceID	In	String
out_VerificationStatus	Out	String

Variable	Type
statusText	String

1. Use Application/Browser — *Display Name: Attach Verification Portal*

Property / Field	Value
Browser	Microsoft Edge
URL	in_PortalUrl
Open	IfNotOpen
Close	Never

1.1. Type Into — *Display Name: Enter Invoice ID*

Property / Field	Value
Target	input id = invoiceId
Text	in_InvoiceID
Empty field	True

1.2. Click — *Display Name: Verify Invoice*

Property / Field	Value
Target	button id = searchBtn

1.3. Check App State — *Display Name: Wait Status*

Property / Field	Value
Target	div id = status
Wait	Appear
Timeout	5 seconds

1.4. Get Text — *Display Name: Get Verification Text*

Property / Field	Value
Target	div id = status
Save to	statusText

1.5. Assign — *Display Name: Normalize Status*

Property / Field	Value
To	out_VerificationStatus
Value	statusText.Replace("STATUS |", "").Trim.ToUpper

4) Workflows\Write_Result.xaml

Argument	Direction	Type
in_OutputExcelPath	In	String
in_TransactionID	In	String
in_InvoiceID	In	String
in_CustomerID	In	String
in_CustomerName	In	String
in_VerificationStatus	In	String
in_OriginalAmount	In	Double
in_Currency	In	String
in_AmountOMR	In	Object
in_ProcessingStatus	In	String

Variable	Type
dtResultRow	DataTable

1. Build Data Table — *Display Name: Build One Result Row*

Property / Field	Value
Columns	Transaction ID(String), Invoice ID(String), Customer ID(String), Customer Name(String), Verification Status(String), Original Amount(Double), Currency(String), Amount OMR(Object), Processing Status(String), Processed At(String)
Save to	dtResultRow

2. Add Data Row — *Display Name: Add Result*

Property / Field	Value
DataTable	dtResultRow
ArrayRow	New Object(){in_TransactionID, in_InvoiceID, in_CustomerID, in_CustomerName, in_VerificationStatus, in_OriginalAmount, in_Currency, in_AmountOMR, in_ProcessingStatus, Now.ToString("dd/MM/yyyy HH:mm:ss")}

3. Use Excel File — *Display Name: Open Generated Report*

Property / Field	Value
Excel file	in_OutputExcelPath

3.1. Append Range — *Display Name: Append Result*

Property / Field	Value
Sheet	Processed Transactions
DataTable	dtResultRow
Exclude headers	True

L. Process.xaml — Exact Transaction Logic

Existing Argument	Keep
in_TransactionItem	QueueItem
in_Config	Dictionary(Of String,Object)

Variable	Type
transactionID	String
invoiceID	String
customerID	String
pdfFileName	String
currency	String
amount	Double
typeValue	String
currentOrPrevious	String
pdfPath	String
pdfCustomerID	String
pdfCustomerName	String
masterCustomerName	String
customerValid	Boolean
validationMessage	String
verificationStatus	String
amountOMR	Double
outputExcelPath	String
dtCustomerMaster	DataTable

1. Assign — *Display Name: Set transactionID*

Property / Field	Value
To	transactionID
Value	in_TransactionItem.SpecificContent("TransactionID").ToString

2. Assign — *Display Name: Set invoiceID*

Property / Field	Value
To	invoiceID
Value	in_TransactionItem.SpecificContent("InvoiceID").ToString

3. Assign — *Display Name: Set customerID*

Property / Field	Value
To	customerID
Value	in_TransactionItem.SpecificContent("CustomerID").ToString

4. Assign — *Display Name: Set pdfFileName*

Property / Field	Value
To	pdfFileName
Value	in_TransactionItem.SpecificContent("PDFFileName").ToString

5. Assign — *Display Name: Set currency*

Property / Field	Value
To	currency
Value	in_TransactionItem.SpecificContent("Currency").ToString.Trim.ToUpper

6. Assign — *Display Name: Set amount*

Property / Field	Value
To	amount
Value	Convert.ToDouble(in_TransactionItem.SpecificContent("Amount"))

7. Assign — *Display Name: Set typeValue*

Property / Field	Value
To	typeValue
Value	in_TransactionItem.SpecificContent("Type").ToString

8. Assign — *Display Name: Set currentOrPrevious*

Property / Field	Value
To	currentOrPrevious
Value	in_TransactionItem.SpecificContent("CurrentOrPrevious").ToString

9. Assign — *Display Name: Set outputExcelPath*

Property / Field	Value
To	outputExcelPath
Value	in_Config("OutputExcelPath").ToString

10. Assign — *Display Name: Set dtCustomerMaster*

Property / Field	Value
To	dtCustomerMaster
Value	DirectCast(in_Config("CustomerMasterDT"), DataTable)

11. If — *Display Name: Validate Current*

Property / Field	Value
Condition	Not currentOrPrevious.Equals("Current", StringComparison.OrdinalIgnoreCase)

THEN. Invoke Write_Result.xaml — *Display Name: Write BRE Row*

Property / Field	Value
Processing Status	"Business Exception: Transaction is not Current"
Amount OMR	Nothing

THEN. Throw — *Display Name: BRE Not Current*

Property / Field	Value
Exception	New BusinessRuleException("Transaction is not Current.")

12. If — *Display Name: Validate Amount*

Property / Field	Value
Condition	amount <= 0

THEN. Invoke Write_Result.xaml — *Display Name: Write BRE Row*

Property / Field	Value
Processing Status	"Business Exception: Amount <= 0"
Amount OMR	Nothing

THEN. Throw — *Display Name: BRE Invalid Amount*

Property / Field	Value
Exception	New BusinessRuleException("Amount must be greater than 0.")

13. Assign — *Display Name: Build PDF Path*

Property / Field	Value
To	pdfPath
Value	Path.Combine(Environment.CurrentDirectory, in_Config("PdfFolder").ToString, pdfFileName)

14. Invoke Workflow File — *Display Name: Read Invoice PDF*

Property / Field	Value
Workflow	Workflows\Read_Invoice_PDF.xaml
in_PdfPath	pdfPath
out_PdfCustomerID	pdfCustomerID
out_PdfCustomerName	pdfCustomerName

15. Invoke Workflow File — *Display Name: Validate Customer*

Property / Field	Value
Workflow	Workflows\Validate_Customer.xaml
in_CustomerMasterDT	dtCustomerMaster
in_QueueCustomerID	customerID
in_PdfCustomerID	pdfCustomerID
in_PdfCustomerName	pdfCustomerName
out_MasterCustomerName	masterCustomerName
out_IsValid	customerValid
out_ErrorMessage	validationMessage

16. If — *Display Name: Customer Validation Failed*

Property / Field	Value
Condition	Not customerValid

THEN. Invoke Write_Result.xaml — *Display Name: Write Customer BRE*

Property / Field	Value
Customer Name	pdfCustomerName
Verification Status	"NOT CHECKED"
Amount OMR	Nothing
Processing Status	"Business Exception: " + validationMessage

THEN. Throw — *Display Name: Customer Business Exception*

Property / Field	Value
Exception	New BusinessRuleException(validationMessage)

17. Invoke Workflow File — *Display Name: Verify Invoice Portal*

Property / Field	Value
Workflow	Workflows\Verify_Invoice_Portal.xaml
in_PortalUrl	in_Config("PortalUrl").ToString
in_InvoiceID	invoiceID
out_VerificationStatus	verificationStatus

18. If — *Display Name: Portal Not Verified*

Property / Field	Value
Condition	Not verificationStatus.Equals("VERIFIED", StringComparison.OrdinalIgnoreCase)

THEN. Invoke Write_Result.xaml — *Display Name: Write Portal BRE*

Property / Field	Value
Customer Name	masterCustomerName
Verification Status	verificationStatus
Amount OMR	Nothing
Processing Status	"Business Exception: Portal status " + verificationStatus

THEN. Throw — *Display Name: Portal Business Exception*

Property / Field	Value
Exception	New BusinessRuleException("Verification Portal returned " + verificationStatus + ".")

19. Switch<String> — *Display Name: Currency Conversion*

Property / Field	Value
Expression	currency
TypeArgument	String

Case "USD". Assign — *Display Name: USD to OMR*

Property / Field	Value
To	amountOMR
Value	amount * Convert.ToDouble(in_Config("FXRate"))

Case "OMR". Assign — *Display Name: OMR No Conversion*

Property / Field	Value
To	amountOMR
Value	amount

Default. Invoke Write_Result.xaml — *Display Name: Write Currency BRE*

Property / Field	Value
Verification Status	verificationStatus
Amount OMR	Nothing

Property / Field	Value
Processing Status	"Business Exception: Unsupported currency " + currency

Default. Throw — *Display Name: Unsupported Currency BRE*

Property / Field	Value
Exception	New BusinessRuleException("Unsupported currency: " + currency)

20. Invoke Workflow File — *Display Name: Write Successful Result*

Property / Field	Value
Workflow	Workflows\Write_Result.xaml
Customer Name	masterCustomerName
Verification Status	verificationStatus
Amount OMR	amountOMR
Processing Status	"Success"

Critical REFramework rule
Do not add a Catch System.Exception around the entire Process.xaml that swallows the exception. Let unexpected exceptions bubble to Main.xaml. The framework then performs the standard System Error path, retry logic, screenshot logic, application re-initialization, and SetTransactionStatus handling.

M. Framework\SetTransactionStatus.xaml / Retry / Screenshot

Change required: NONE
Keep SetTransactionStatus.xaml. It already marks a QueueItem Successful when Process.xaml completes, Failed with ErrorType Business when BusinessRuleException is thrown, and Failed with ErrorType Application for system errors. It also updates transaction/retry counters. Keep TakeScreenshot.xaml and the framework retry logic.

Retry configuration
Because this final solution uses Orchestrator Queue Auto Retry = 1, set MaxRetryNumber = 0 in Config.xlsx to avoid two independent retry mechanisms. Keep MaxConsecutiveSystemExceptions = 3 and ShouldMarkJobAsFaulted = TRUE if those standard rows are present in the generated template.

N. Workflows\Finalize_Run.xaml

Argument	Direction	Type
in_Config	In	Dictionary(Of String,Object)

Variable	Type
outputPath	String
dtResults	DataTable
dtActual	DataTable
successCount	Int32
businessExceptionCount	Int32
totalOMR	Double
averageOMR	Double

1. If — *Display Name: Check Output Exists*

Property / Field	Value
Condition	in_Config IsNot Nothing AndAlso in_Config.ContainsKey("OutputExcelPath") AndAlso File.Exists(in_Config("OutputExcelPath").ToString)

THEN.1. Assign — *Display Name: Set Output Path*

Property / Field	Value
To	outputPath
Value	in_Config("OutputExcelPath").ToString

THEN.2. Use Excel File — *Display Name: Read Final Results*

Property / Field	Value
Excel file	outputPath

THEN.2.1. Read Range — *Display Name: Read Processed Transactions*

Property / Field	Value
Sheet	Processed Transactions
Range	A3:J100
Has headers	True
Save to	dtResults

THEN.3. Assign — *Display Name: Remove Blank Rows*

Property / Field	Value
To	dtActual
Value	dtResults.AsEnumerable().Where(Function(r) Not String.IsNullOrWhiteSpace(r("Transaction ID").ToString)).CopyToDataTable()

THEN.4. Assign — *Display Name: Count Success*

Property / Field	Value
To	successCount
Value	dtActual.AsEnumerable().Count(Function(r) r("Processing Status").ToString.Equals("Success", StringComparison.OrdinalIgnoreCase))

THEN.5. Assign — *Display Name: Count Business Exceptions*

Property / Field	Value
To	businessExceptionCount
Value	dtActual.AsEnumerable().Count(Function(r) r("Processing Status").ToString.StartsWith("Business Exception", StringComparison.OrdinalIgnoreCase))

THEN.6. Assign — *Display Name: Total OMR*

Property / Field	Value
To	totalOMR
Value	dtActual.AsEnumerable().Where(Function(r) r("Processing Status").ToString.Equals("Success", StringComparison.OrdinalIgnoreCase)).Sum(Function(r) Convert.ToDouble(r("Amount OMR")))

THEN.7. Assign — *Display Name: Average OMR*

Property / Field	Value
To	averageOMR
Value	If(successCount = 0, 0.0, totalOMR / successCount)

THEN.8. Use Excel File — *Display Name: Write Summary*

Property / Field	Value
Excel file	outputPath

THEN.8.1. Write Cell — *Display Name: Write Summary Value*

Property / Field	Value
Value	successCount
Destination	Summary!B4

THEN.8.2. Write Cell — *Display Name: Write Summary Value*

Property / Field	Value
Value	businessExceptionCount
Destination	Summary!B5

THEN.8.3. Write Cell — *Display Name: Write Summary Value*

Property / Field	Value
Value	totalOMR
Destination	Summary!B6

THEN.8.4. Write Cell — *Display Name: Write Summary Value*

Property / Field	Value
Value	averageOMR
Destination	Summary!B7

THEN.9. Use Desktop Outlook App — *Display Name: Use Outlook*

Property / Field	Value
Account	Default profile

THEN.9.1. Send Email — *Display Name: Send Final Report*

Property / Field	Value
To	in_Config("EmailTo").ToString
Subject	in_Config("EmailSubject").ToString
Body	"Please find the final REFramework training report attached."
Attachments	New String() {outputPath}

O. Main.xaml End Process — One Addition

Where to add it

In Main.xaml, inside the existing End Process state, add Invoke Workflow File → Workflows\Finalize_Run.xaml and pass in_Config = Config. Place it before final application cleanup if you want the email while Outlook/browser resources are still available, or immediately after graceful close if Outlook is independent. Keep all existing framework End

Process / ShouldMarkJobAsFaulted logic unchanged.

P. Expected Final Outcome

Transaction	Expected Status	Amount OMR / Reason
FNL-TX-001	Success	303.188
FNL-TX-002	Success	356.125
FNL-TX-003	Success	1062.500
FNL-TX-004	Success	462.000
FNL-TX-005	Business Exception	Portal status ON HOLD
FNL-TX-006	Success	1475.000
FNL-TX-007	Success	620.813
FNL-TX-008	Business Exception	PDF Customer Name mismatch
FNL-TX-009	Success	1887.500
FNL-TX-010	Business Exception	Portal status ON HOLD
FNL-TX-011	Success	832.563
FNL-TX-012	Success	2300.000

Summary Metric	Expected
Successful Transactions	9
Business Exceptions	3
Total OMR	9299.688
Average OMR	1033.299

Q. REFramework Reference Notes

- Current UiPath Studio documentation: Robotic Enterprise Framework remains a State Machine project template with prebuilt states for initialization, input/transaction retrieval, processing, and exception handling.
- Official UiPath/ReFrameWork source repository (archived 26 June 2026): confirms Main.xaml State Machine, queue-based Framework/GetTransactionData.xaml, root Process.xaml, and Framework/SetTransactionStatus.xaml behavior.
- UiPath REFramework release improvements introduced OrchestratorQueueName, stronger queue retries, MaxConsecutiveSystemExceptions / ConsecutiveSystemExceptions, ShouldMarkJobAsFaulted, screenshot information, and stricter asset initialization behavior.
- REFramework configuration convention: Settings and Constants are read into Config; Assets maps a Config key to an Orchestrator asset value. Credentials should not be stored as plain text in Config.xlsx.

Final Build Checklist

Scenario	Before Marking Complete
01	14 IT Approved records; Total 3131.000; Average 223.643
02	12 Process=Yes rows updated; No rows untouched
03	Customer + all expected billing values match table
04	New timestamped workbook created; template unchanged; email attaches generated file
05	35 queue items; 28 success; 7 business exceptions; Currency uses Switch
06	REFramework states retained; queue status logic not duplicated; 9 success / 3 BRE; Total OMR 9299.688